Generating data Activity Book

Data Modeling for Qlik Sense

© 2023 QlikTech International AB. All rights reserved. All company and/or product names may be trade names, trademarks and/or registered trademarks of the respective owners with which they are associated.

DMQS May 2023

Table of Contents

Generating data Activity Book

Table of Contents

Activity 1: Create new fields for Orders and OrderDetails

Challenge 3
Hints 3
Solution 5

Activity 2: Where clause in Orders table

Challenge 6
Hints 6
Solution 7

Activity 3: Resident load SalesPerson table

Challenge 8
Hints 8
Solution 10

Activity 4: Use a Where Exists() clause

Challenge 11
Hints 11
Solution 12

Activity 5: Using compound keys to solve model issues

Challenge 12
Hints 13
Part 1 - Load the Shipments table 13
Progress check 14
Part 2 - Create a composite key 14
Progress check 16
Part 3 - Eliminate the second synthetic key 16
Solution 17

Activity 7: Use flags

Challenge 17
Hints 18
Solution 19

Activity 7: Use flags

Challenge 19
Hints 20
Solution 21

Activity 1: Create new fields for Orders and OrderDetails

Summary:

- Generate additional data values in the orders table for day, month and year of orders.
- Create a calculated expression in the OrderDetails table for the exact sales amount of an order line.

Qlik Sense app: ABC Sales Data

Challenge

Transform your existing data model by creating new calculated fields:

- In the *Orders* table, modify the script and create the following fields to allow for the analysis of trends over year, month and day:
  - *Year*, that contains the year part of the *OrderDate* field.
  - *Month*, that contains the month of the *OrderDate* field.
  - *Day*, that contains the day of the *OrderDate* field.
- In the *OrderDetails* table, create the following field that allows for the analysis of the company's orders:
  - *LineSalesAmount*, a field that contains the total sales amount for every order line.
- Load your data and check your data model to verify the changes.

Hints

- The tables where the fields must be added are in the **Orders** section of the **Data load editor**.
- The date functions **Year()**, **Month()**, and **Day()** must be used to create the fields.
- The expression for the *LineSalesAmount* field is the following: `UnitPrice * Quantity * (1 - Discount)`.

Step-by-step guide

1. In the **Orders** section, within the *Orders* table **LOAD**, after the *OrderDate* field, enter a carriage return to make space for new script. Make sure there is a comma right after the previous field.
2. Add the following three lines of loading script and make sure you use the correct syntax at the end:

```
YEAR(OrderDate) AS Year,
MONTH(OrderDate) AS Month,
DAY(OrderDate) AS Day;
```

3. In the *OrderDetails* table **LOAD**, after *UnitPrice*, enter a carriage return to make space for new script.
4. Add the following line of script and make sure you use the correct syntax at the end:

```
UnitPrice * Quantity * (1 - Discount) AS LineSalesAmount,
```

Note: In the preceding line, the spaces on either side of the * and - operators are optional, but are included for readability.

The **Orders** script section now looks as follows:

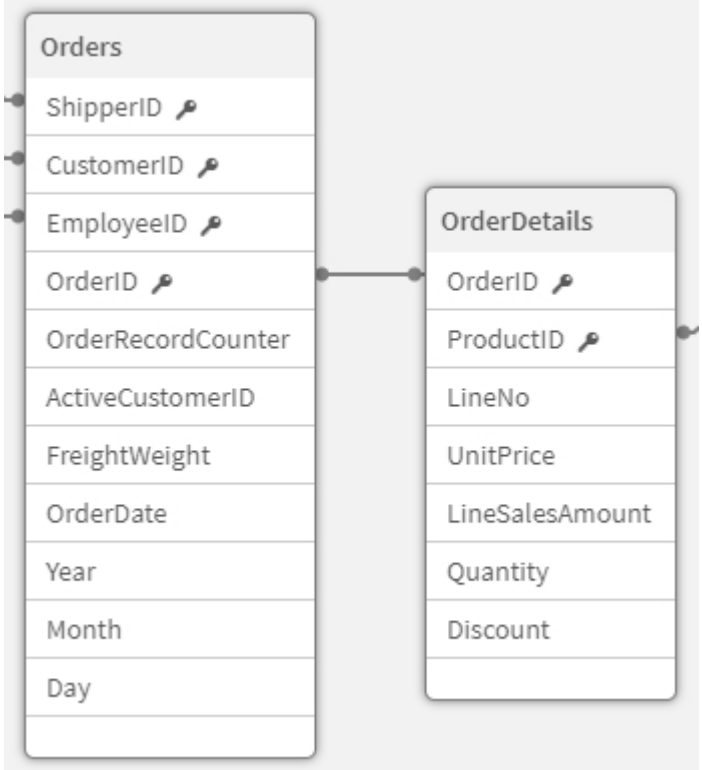
```
LIB CONNECT TO 'ABC';

// **** Load Orders table ****
[Orders]:
LOAD OrderID,
    //  OrderID AS OrderRecordCounter,
    1 AS OrderRecordCounter,
    CustomerID,
    CustomerID AS ActiveCustomerID,
    EmployeeID,
    ShipperID,
    FreightWeight,
    OrderDate,
    YEAR(OrderDate) AS Year,
    MONTH(OrderDate) AS Month,
    DAY(OrderDate) AS Day;
SELECT OrderID,
    CustomerID,
    EmployeeID,
    ShipperID,
    FreightWeight,
    OrderDate
FROM "ABC_2025".dbo.orders;


// **** Load OrderDetails table ****
[OrderDetails]:
LOAD OrderID,
    LineNo,
    ProductID,
    UnitPrice,
    UnitPrice * Quantity * (1 - Discount) AS LineSalesAmount,
    Quantity,
```

```
    Discount;
SELECT OrderID,
    "LineNo",
    ProductID,
    UnitPrice,
    Quantity,
    Discount
FROM "ABC_2025".dbo.orderdetails;
```

Copy to clipboard

5. Load your data after you apply the changes and navigate to the **Data model viewer**.
6. Confirm that the new fields have been added. You might need to expand the tables in order to view all the fields.

Solution



Activity 2: Where clause in Orders table

Summary:

- Alter the script which loads the Orders table to restrict the data extracted, based on the order date.
- Analyze the resulting year values within the sheet objects.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, use a condition to determine specific records for loading from the source by doing the following:

- In the *Orders* table, add a **WHERE** clause to load records of specific date range.
- In the *OrderDetails* table, add a **WHERE EXISTS** clause to only load records related to the above amount of years.
- Load your data and check the *Year* values in the **Basic data transformations** sheet.

Hints

- The **WHERE** clause in both cases is added at the end of the preceding **LOAD** statement.

Step-by-step guide

1. In the **Data load editor**, navigate to the **Orders** section.
2. In the *Orders* table only, limit the number of orders till the latest date: Add a **WHERE** clause in the **LOAD** statement of the *Orders* table, before the semicolon:

```
WHERE OrderDate < NOW();
```

> **Note 1:** One can argue that only the existing orders would be displayed and not any future order, even without the **WHERE** clause. But, by design, this is training material that has included future order dates. Hence, for capping the date range, the **WHERE** clause is used.

3. In the *OrderDetails* table, there is no *OrderDate* field. So, in the *OrderDetails* table, add a **WHERE EXISTS** clause with *OrderID* in the **LOAD** statement, before the semicolon, to limit the order records that exist in the *Orders* table. This will retrieve *OrderDetails* records that match the loaded date-restricted *Orders*:

```
WHERE EXISTS(OrderID, OrderID);
```

4. Be careful with punctuation. The final fields in the preceding **LOAD** lists should not have a comma or semicolon after them and lead straight into the **WHERE** statements.

> **Note 2:**The first *OrderID* is the existing field in the data model. This field exists in the *Orders* table. The second *OrderID* is the new field being loading from the *OrderDetails* table. During the load, every value of the *OrderID* field in the *OrderDetails* table, is cross-checked against all the values in the existing *OrderID* field in the *Orders* table. If the same

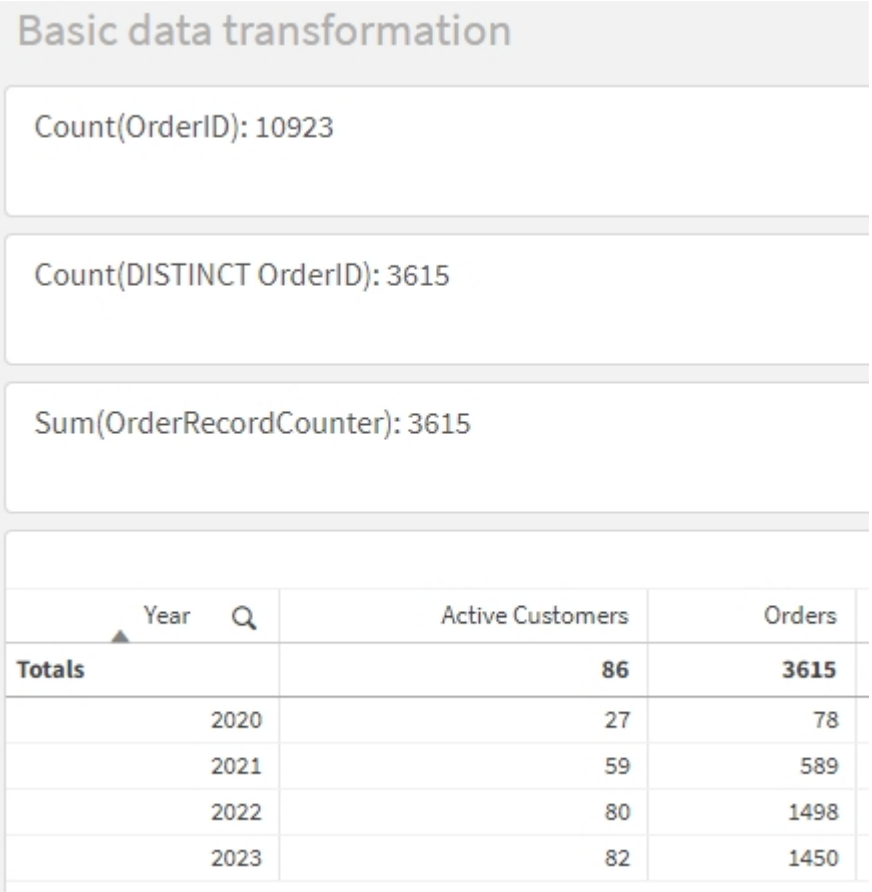
value exists in both fields, the function (*Boolean*) returns **True** and the record is imported into the *OrderDetails* table. If the same *OrderID* value does not exist, the record is omitted.

Note 3: The load order is important and matters when using the **Where Exists** clause. In this case, the *Orders* table needs to be loaded before the *OrderDetails* table in the script for this **Where** clause to work.

Note 4: The **Where Exists** clause in this case uses two parameters. Although this is best practice, one parameter can be used instead.

5. Load your data and navigate to the **Basic data transformations** sheet.
6. To more easily verify that the table now contains the data of the specified date range, edit the sheet and select the **Table** object to open up the **Properties** panel. Within the **Data** section, change the *DivisionName* field, to use the *Year* field instead.
7. Exit edit mode and evaluate the data table.

Solution



Year	Active Customers	Orders
Totals	**86**	**3615**
2020	27	78
2021	59	589
2022	80	1498
2023	82	1450

Activity 3: Resident load SalesPerson table

Summary:

- Use an existing table as a source of data for a new table.
- Restrict the values in the new table with a Where clause.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, create a new **RESIDENT** table by reusing existing data from the *Employees* table. To achieve that, follow the process below:

- In the **Employees** script section, locate the *Employees* table and create a *Name* field by concatenating the fields *First Name* and *Last Name*.
- Create a new script section after **Suppliers** and name it *SalesPerson*.
- Create a new table using **RESIDENT LOAD** that retrieves data from the *Employees* table with the following fields:
  - *EmployeeID*
  - *Name AS SalesPerson*
  - *JobTitle* AS *SalesTitle*
- Add a **WHERE** clause to filter the number of records you are loading in the new table to only those whose title contains the words "*Sales*" or "*Account*".
- Name the new table *SalesPerson* and add a comment to describe the table.
- Load your data and verify that the new table has been created.
- Create a new *Employees* sheet to validate your data.
- In the new sheet, add the fields *Name*, *JobTitle* and *SalesTitle* as **Filter panes**. Select all the *SalesTitle* values.
- Verify that only a part of the *JobTitle* values is associated to your selections because only a subset of the *Employees* table is loaded in the *SalesPerson* table.

Hints

- Only the *Employees* records that satisfy the criteria defined with the **WHERE** clause are added to the *SalesPerson* table.

Step-by-step guide

1. In the **Data load editor**, navigate to the **Employees** section and locate the *Employees* table.
2. Replace the *First Name* and *Last Name* fields with the following line:

```
...
"First Name" & ' ' & "Last Name" AS Name,
...
```

3. Create a new section between **Suppliers** and **Shipments**. Name it *SalesPerson*.
4. Create a new **LOAD** statement to make a table for *SalesPerson* based on the *Employees* table. You cannot do this using **Select data** from a data connection; you need to type in the code. Add the following statement (with comment and label) to your script.

```
// **** Load SalesPerson from Employees ****
SalesPerson:
LOAD
    EmployeeID,
    Name AS SalesPerson,
    JobTitle AS SalesTitle
RESIDENT Employees;
```

Copy to clipboard

5. Limit the load of all *Employees* records to just those you can identify as sales people. To do this, add a **WHERE** condition after the **RESIDENT** line, but before the semicolon, as follows:

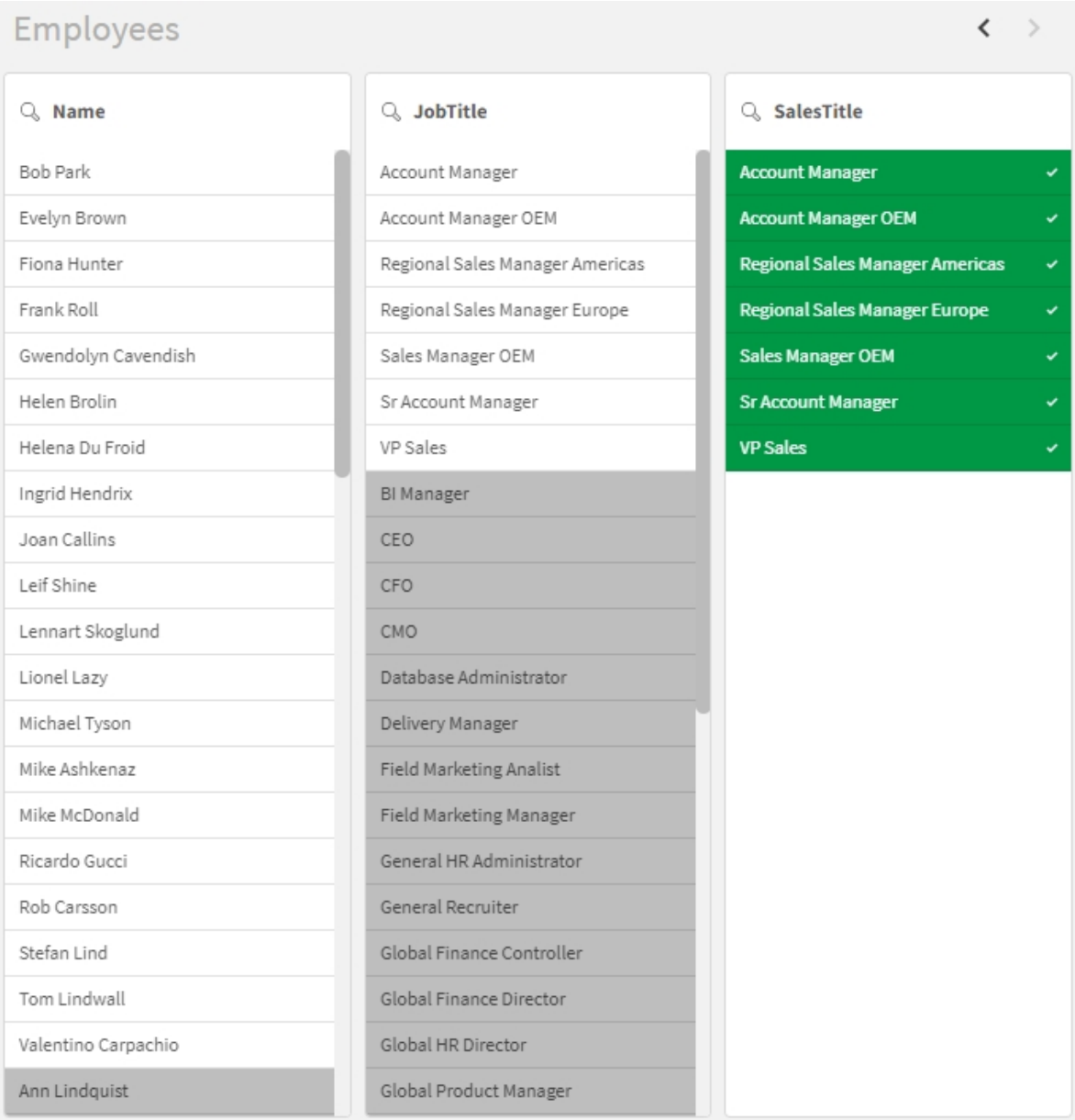
```
WHERE
     JobTitle LIKE '*Sales*'
OR JobTitle LIKE '*Account*';
```

Copy to clipboard

Note: The **LIKE** operator works with the wildcard (*), searches the field, and finds all values that contains the text *Sales* or *Account*.

6. Load your data and verify that the new table has been created.
7. Create a new sheet in the app and name it *Employees*.
8. Open the new **Employees** sheet and enable the edit mode.
9. Add the dimension *Name* to your sheet as a **Filter pane**. Resize it so it is **8 columns** wide and place it on the left of the sheet.
10. Create two more **Filter panes** for *JobTitle* and *SalesTitle* dimensions. Make them the same width as *Name*.
11. Disable edit mode and select all values from the *SalesTitle* filter pane to review the results. Notice that only specific *Name* and *JobTitle* values are shown as possible values associated with the selection. To include a value from the original *Employees* table in the new logical *SalesPersons* table, at least one of the defined conditions for the *JobTitle,* containing either the text "*Sales*" or "*Account*", must be satisfied.

Solution



Activity 4: Use a Where Exists() clause

Summary:

- Modify the SalesPerson table restriction to use only EmployeeIDs which exist from the Orders table.
- Analyze the results on the table visualization.

Qlik Sense app: ABC Sales

Challenge

In the *SalesPerson* table of the **ABC Sales Data** app, modify the **WHERE** clause in the **LOAD** statement to include those records from the *Employees* table that are related to sales. To achieve that, do the following:

- In the **Orders** script section, locate the *Orders* table and add the *EmployeeSalesID* field that contains the same values as the *EmployeeID*.
- In the *SalesPerson* table, replace the previous **WHERE** statement to only include records from the *Employees* table, where the *EmployeeID* exists in the *EmployeeSalesID* field.
- Load your data and check the **Employees** sheet to see the difference.

Hints

- The **EXISTS** function can be used with one or two parameters.

Step-by-step guide

1. In the **Data load editor**, navigate to the **Orders** section and locate the *Orders* table.
2. Add the following script line in the *Orders* table, immediately after the loading of the *EmployeeID* field:

```
EmployeeID AS EmployeeSalesID,
```

3. Locate the *SalesPerson* table in the **SalesPerson** section. Comment out the existing **WHERE** clause statement, and replace it with a new **WHERE** condition using the **EXISTS** function as seen below:

```
// **** Load SalesPerson from Employees ****
SalesPerson:
LOAD
    EmployeeID,
    Name AS SalesPerson,
    JobTitle AS SalesTitle
RESIDENT Employees
WHERE EXISTS(EmployeeSalesID, EmployeeID);
// WHERE JobTitle LIKE '*Sales*'
// OR JobTitle LIKE '*Account*';
```

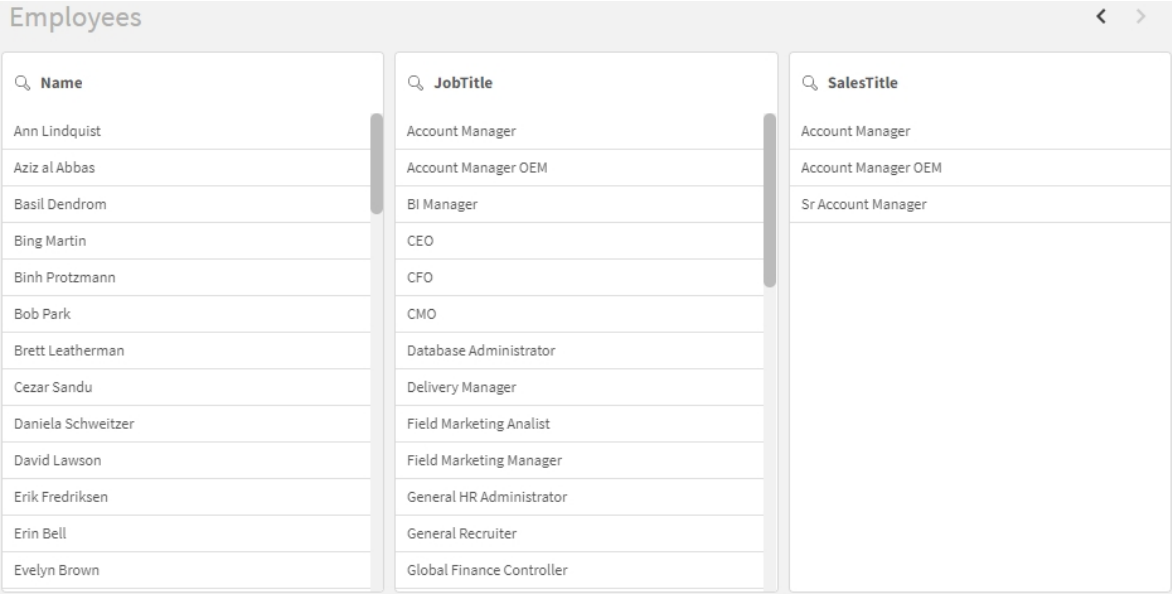
Copy to clipboard

The condition in the **WHERE** clause checks that the loaded data has matching values in the *EmployeeSalesID* field.

Be mindful of the order of your script statements. The reference field must be populated prior to checking for values.

4. Load your data and navigate to the **Employees** app sheet. Notice that only three values appear for *SalesTitle* and the possible values for *JobTitle* and *Name* have also reduced. Why is this?

Solution



Activity 5: Using compound keys to solve model issues

Summary:

- Load shipments data from the ABC_2025 database and review the issues caused in the data model.
- Create compound keys in the Shipments and OrderDetails sections to provide one, single field to associate the tables upon.
- Comment out additional fields which cause synthetic keys and circular references.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, load a new table and observe the issues in your data model. To achieve this:

- Navigate to the end of the **Shipments** script section.
- Load the *shipments* table from the **ABC_2025** database and label it *Shipments*. Add appropriate commentary.
- Load your data and check the data model to verify that the table has been loaded; observe the issues that have appeared in your data model.
- Navigate to the **Shipments** script section and locate the *Shipments* table.
- Create a composite key based on *OrderID* and *LineNo*:
  `OrderID &'|'& LineNo AS OrderLineKey,`
- Comment out any fields from tables that you no longer need.

- Copy the composite key you just created to the *OrderDetails* table to create a new association.
- Load your data and verify that the synthetic key between the *Shipments* and the *OrderDetails* tables is gone.
- In the *Shipments* table, use a **WHERE EXISTS** clause to limit the records you are loading to only those which contain *OrderID* in the dataset.
- In order to optimize the script, use the **AutoNumber** function with the composite keys you created for both *Shipments* and *Orders* tables. Observe the difference by previewing the *OrderLineKey* field.
- In the *Shipments* table of your script, comment out the fields that are repeated in the *Orders* table. Leave only the *OrderLineKey* key and the *ShipmentDate*, as they are the only fields you need to keep in the data model.
- Load your data and verify there are no more issues in your data model.

Hints

- Three synthetic keys have been created in your data model because the new table has multiple fields in common with *Orders* and *OrderDetails* tables.
- Comment out the fields *OrderID*, *LineNo* and *ProductID* from the *Shipments* table ONLY as they are no longer needed after the composite key.
- The following three fields must be commented out from the *Shipments* table: *CustomerID*, *EmployeeID*, and *ShipperID*. These fields also exist in the *Orders* table, so the data is available through the associations.
- Eliminating the synthetic key will also eliminate the loop (circular reference) in your data model.

Part 1 - Load the Shipments table

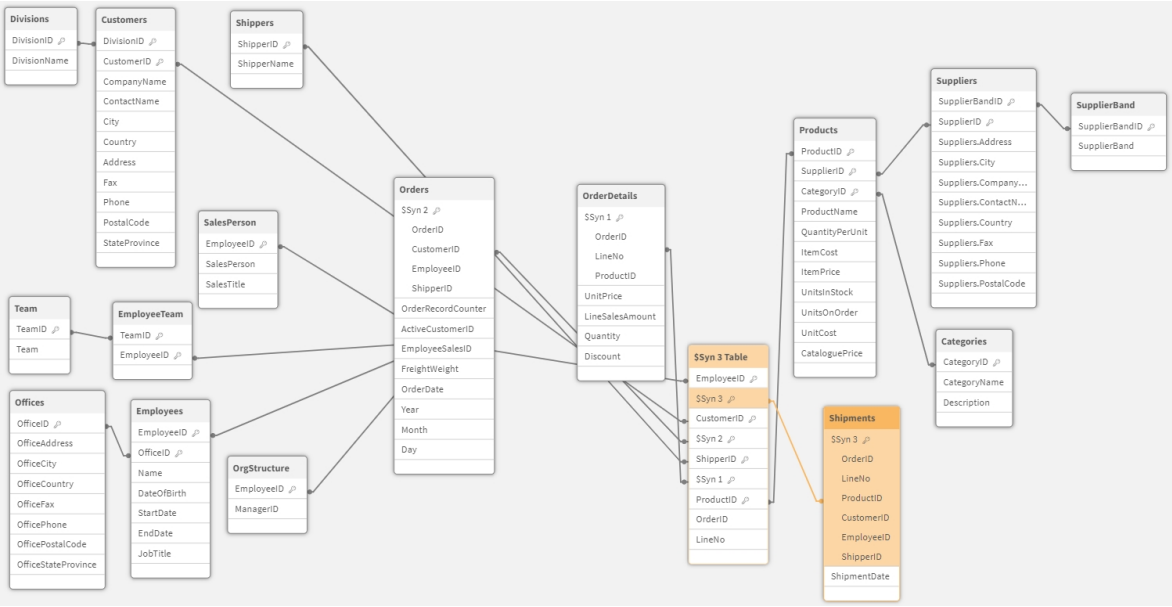
Step-by-step guide

1. Navigate to the end of the **Shipments** script section.
2. Use the **ABC** database connection to load the *shipments* table and make sure all the fields are selected.
3. Add a suitable comment before the **LOAD** statement and relabel the table *[shipments]:* whilst capitalizing the "S".
4. Remove the **LIB CONNECT** statement between the *Shippers* and *Shipments* table loads.
5. Load the data and examine the synthetic key warnings.
6. In the **Data model viewer**, notice a synthetic table was generated. The *$Syn 3 Table* contains 3 synthetic keys with common fields between the *OrderDetails*, *Orders*, and *Shipments*. There is one synthetic key for each table because the common fields vary:
    - *$Syn 1* associates the synthetic table with *OrderDetails*. The common fields are: *OrderID, LineNo,* and *ProductID*.
    - *$Syn 2* associates the synthetic table with *Orders*. The common fields are: *CustomerID, EmployeeID, ShipperID,* and *OrderID*.
    - *$Syn 3* associates the synthetic table with *Shipments*. The common fields are: *OrderID, LineNo, ProductID, CustomerID, EmployeeID,* and *ShipperID*.

In the scenario you are currently working on, there is a business rule that says each product can be shipped separately. So, even if you have products that belong to the same *OrderID*, they can still be shipped as individual items.

This means that there is a one-to-one relationship between *OrderDetails* and *Shipments*. The field *ProductID* gets the values directly from *OrderDetails*. There is no need to have it twice in the data model. Therefore, remove the *ProductID* field from *Shipments*, in the next part of the activity.

Progress check



Click on the Progress check image to open as a pop-up, or right-click on the image for more options. To close, click on the image again.

Part 2 - Create a composite key

Step-by-step guide

1. Navigate to the **Shipments** script section and locate the *Shipments* table.
2. Create a composite key field, based on *OrderID* and *LineNo* using “&” to concatenate the fields and the pipe (|) as separator. Name the key field *OrderLineKey*:

```
OrderID & '|' & LineNo AS OrderLineKey,
```

3. Comment out the fields *OrderID*, *LineNo*, and *ProductID* in the *Shipments* table. Since you have now found an alternative way to associate the two tables, there is no point of keeping those fields twice in the data model. This is necessary to avoid the synthetic key. Those fields still remain in the *OrderDetails* table.
4. Copy the composite key you just created and paste it to the *OrderDetails* table. Leave the fields *OrderID* and *LineNo* unchanged.
5. Load the data and observe the warning messages in the load progress. You get a warning for one synthetic key and one error due to a circular reference because one or more loops have

been found in the data model. Notice that before, there were 3 synthetic keys. Now, there is just one remaining.

6. Switch to **Analyze** mode, create a new sheet and name it *Composite key*.
7. Enable edit mode on the new sheet.
8. Add the following dimensions to the sheet as **Filter panes**: *OrderID*, *LineNo*, and *OrderLineKey*.
9. Disable the edit mode in the sheet and select one value in the *OrderID* field. Review the associated values (white) in the *OrderLineKey* field.
10. Clear the selection. Choose an item from the top of the *OrderLineKey* filter pane, e.g. 1|1. There are some values in the field that do not associate with any *OrderID* and *LineNo*. Why is this? In a previous activity, you used a **WHERE** clause in the *Orders* and *OrderDetails* tables. The *Shipments* table loads all the orders instead.
11. Clear the selection and return to the **Data load editor**.
12. In the **Shipments** script section, add the following **WHERE** clause before the semicolon in the preceding **LOAD** statement:

```
WHERE EXISTS(OrderID, OrderID);
```

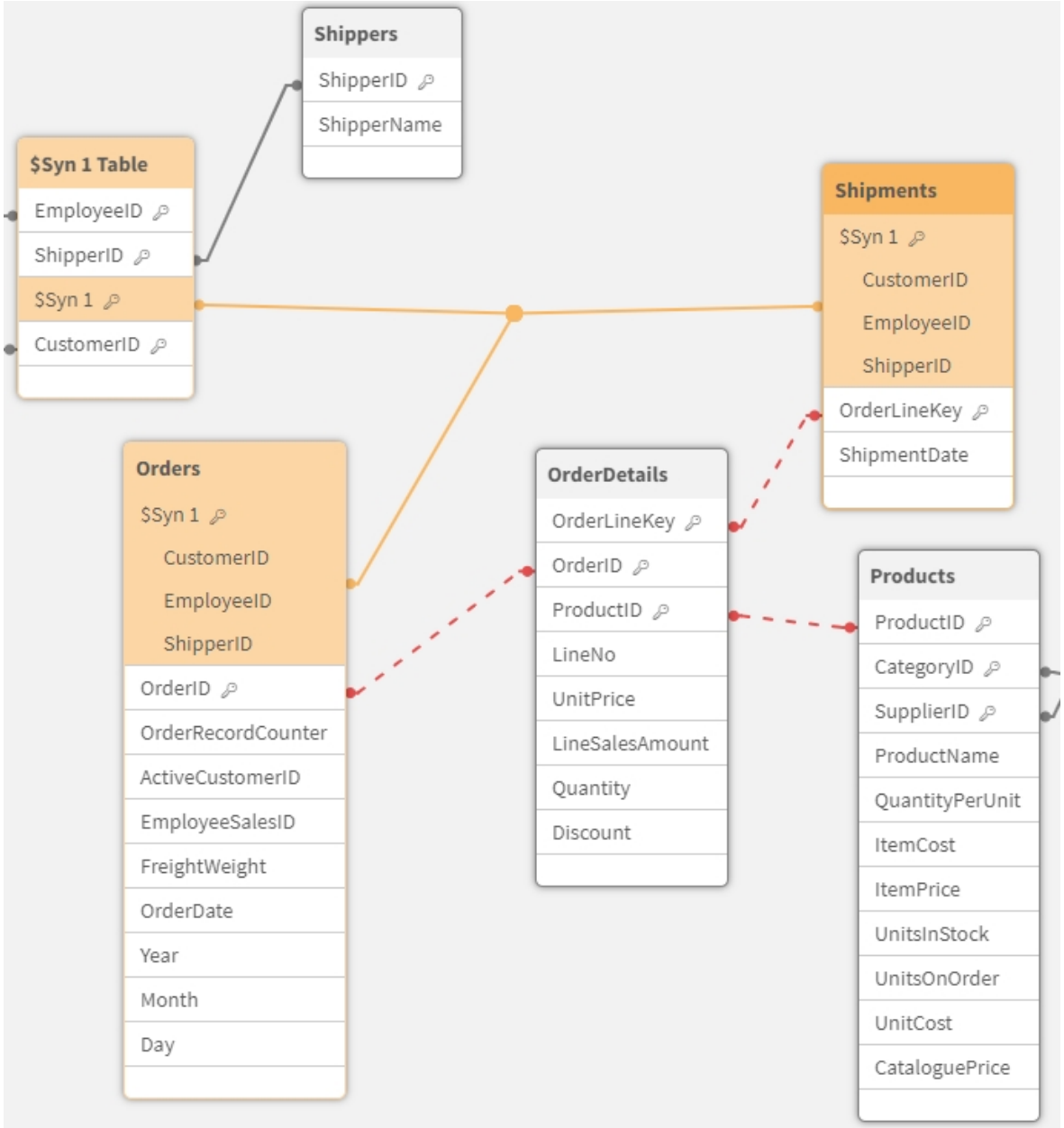
13. Apply the **AutoNumber** function to get a numeric representation for the *OrderLineKey* field. In the **Data load editor**, change the *OrderLineKey* line in the *OrderDetails* and *Shipments* tables to match the following script:

```
AUTONUMBER(OrderID & '|' & LineNo, 'OrderLine') AS OrderLineKey,
```

Note the second parameter in this function; *OrderLine* is optional in this case, as there are no other composite keys using the **AutoNumber** function. It is added here as a best practice.

14. Load the data again and open the **Composite key** sheet. Check the values in the *OrderLineKey* field. They are now sequential numeric values.

Progress check



Click on the Progress check image to open as a pop-up, or right-click on the image for more options. To close, click on the image again.

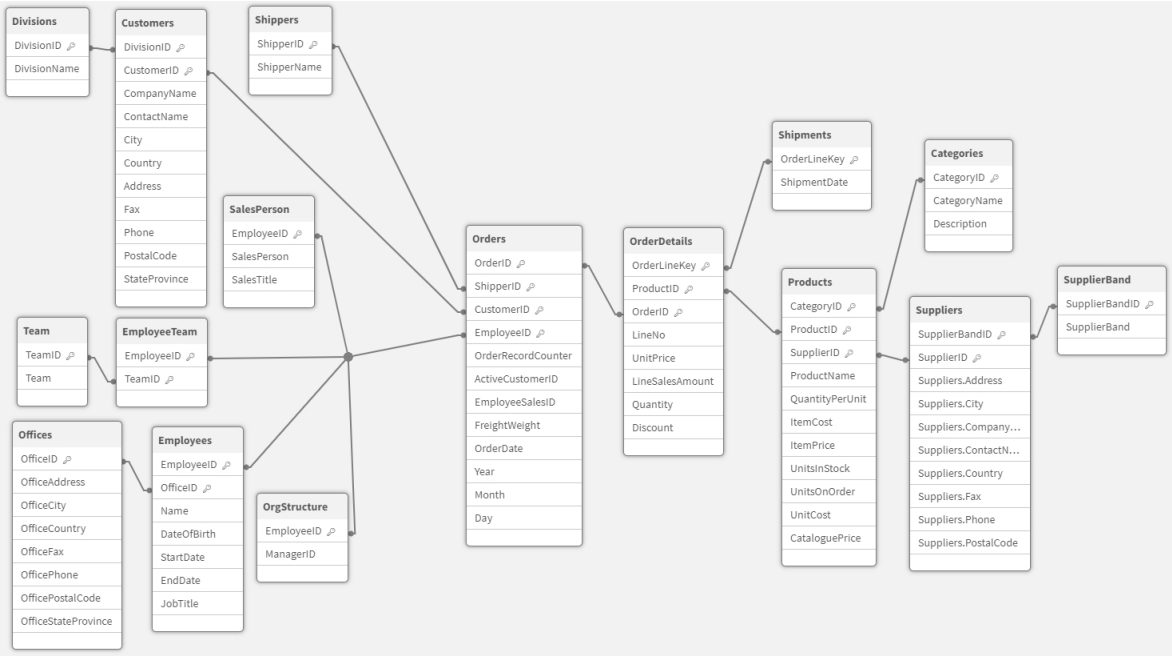
Part 3 - Eliminate the second synthetic key

Step-by-step guide

1. In the **Data model viewer**, observe your data model. The synthetic key and circular reference exist because there is more than one common field between the tables *Shipments* and *Orders*. In addition, there are connections from *Orders* to *OrderDetails* to *Shipments* to *$Syn 1 Table*, and back to *Orders* again.

2. Navigate to the **Shipments** script section and comment out the three fields *CustomerID, EmployeeID,* and *ShipperID* from the *Shipments* table.
3. Ensure that the last non-commented field in the **LOAD** statement, *ShipmentDate*, is not followed by a comma.
4. Load your data and notice that there are no warnings or errors during the reload.
5. In your data model, verify that the circular reference and the synthetic key have been eliminated.

Solution



Activity 7: Use flags

Summary:

- Create a new app sheet and table object.
- Add one dimension and three measures.
- Evaluate the three measures which utilize the calendar flags.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, test two of the flags you created in the *MasterCalendar* by completing the following:

- Create a new app sheet and name it *Calendar Flags*.
- In the new sheet, create a table with *CompanyName* as dimension.

- In the same table, add the following expressions as measures:
  - CYTD: *SUM(LineSalesAmount * CYTDFlag)* that calculates the current year-to-date sales.
  - LYTD: *SUM(LineSalesAmount * LYTDFlag)* that calculates the last year-to-date sales.
  - *CYTD - LYTD* that calculates the difference.
- Change the number format to *Money* for all the above expressions.

Hints

- The flag fields created in the *MasterCalendar* table only accept two values: *0* and *1*.
- The column labels can also be used as part of a calculation expression.

Step-by-step guide

1. Create a new app sheet and name it *Calendar Flags*.
2. Enable edit mode, insert a **Table** to cover the sheet and add the field *CompanyName* as a dimension.
3. From the **Properties** panel, add a measure with the following expression:
   `SUM(LineSalesAmount * CYTDFlag)`
4. Label the new measure *CYTD* and change the number format to *Money*.

   Note: When using **Money** as the **Number formatting**, Qlik Sense utilizes the settings from the data load script to determine the format pattern. If required, it can be overridden.

5. Add a second measure column, using the expression:
   `SUM(LineSalesAmount * LYTDFlag)`
6. Label the second measure *LYTD* and change the number format to *Money*.
7. Add a third measure column, using the expression:
   `CYTD - LYTD`

   Note: *CYTD* and *LYTD* are not reserved Qlik Sense words. When used as part of an expression, the **Expression editor** might give a *Bad field name: CYTD, LYTD* warning. Despite the warning, the expression still works as these names refer to the column names you previously added. You can ignore the warning and continue to the next step.

8. Label the third column *CYTD vs LYTD* and change the number format to *Money* again.
9. Disable the edit mode in the sheet and clear your selections, if any.

Solution

Calendar Flags

CompanyName	CYTD	LYTD	CYTD vs LYTD
Totals	**$ 4322798.15**	**$ 2206015.92**	**$ 2116782.23**
Alles Lusekofter	$ 0.00	$ 0.00	$ 0.00
Art et Fashion	$ 53615.90	$ 26924.58	$ 26691.31
Aujourd´hui	$ 12060.55	$ 5231.77	$ 6828.78
Autokleider	$ 24303.05	$ 17044.67	$ 7258.38
Belgium Black Jeans	$ 44726.48	$ 0.00	$ 44726.48
Big Foot Shoes	$ 21255.19	$ 11606.34	$ 9648.85
Boleros	$ 214752.71	$ 191627.32	$ 23125.39
Bond Ltd	$ 95338.61	$ 51400.51	$ 43938.10
Boombastic	$ 119177.71	$ 58992.75	$ 60184.97
Casual Clothing	$ 45651.67	$ 17723.93	$ 27927.74
Champes	$ 120196.44	$ 114127.58	$ 6068.87

Activity 7: Use flags

Summary:

- Create a new app sheet and table object.
- Add one dimension and three measures.
- Evaluate the three measures which utilize the calendar flags.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, test two of the flags you created in the *MasterCalendar* by completing the following:

- Create a new app sheet and name it *Calendar Flags*.
- In the new sheet, create a table with *CompanyName* as dimension.
- In the same table, add the following expressions as measures:
  - CYTD: *SUM(LineSalesAmount * CYTDFlag)* that calculates the current year-to-date sales.
  - LYTD: *SUM(LineSalesAmount * LYTDFlag)* that calculates the last year-to-date sales.
  - *CYTD - LYTD* that calculates the difference.
- Change the number format to *Money* for all the above expressions.

Hints

- The flag fields created in the *MasterCalendar* table only accept two values: *0* and *1*.
- The column labels can also be used as part of a calculation expression.

Step-by-step guide

1. Create a new app sheet and name it *Calendar Flags*.
2. Enable edit mode, insert a **Table** to cover the sheet and add the field *CompanyName* as a dimension.
3. From the **Properties** panel, add a measure with the following expression:
   `SUM(LineSalesAmount * CYTDFlag)`
4. Label the new measure *CYTD* and change the number format to *Money*.

   Note: When using **Money** as the **Number formatting**, Qlik Sense utilizes the settings from the data load script to determine the format pattern. If required, it can be overridden.

5. Add a second measure column, using the expression:
   `SUM(LineSalesAmount * LYTDFlag)`
6. Label the second measure *LYTD* and change the number format to *Money*.
7. Add a third measure column, using the expression:
   `CYTD - LYTD`

   Note: *CYTD* and *LYTD* are not reserved Qlik Sense words. When used as part of an expression, the **Expression editor** might give a *Bad field name: CYTD, LYTD* warning. Despite the warning, the expression still works as these names refer to the column names you previously added. You can ignore the warning and continue to the next step.

8. Label the third column *CYTD vs LYTD* and change the number format to *Money* again.
9. Disable the edit mode in the sheet and clear your selections, if any.

Solution

Calendar Flags

CompanyName	CYTD	LYTD	CYTD vs LYTD
Totals	**$ 4322798.15**	**$ 2206015.92**	**$ 2116782.23**
Alles Lusekofter	$ 0.00	$ 0.00	$ 0.00
Art et Fashion	$ 53615.90	$ 26924.58	$ 26691.31
Aujourd´hui	$ 12060.55	$ 5231.77	$ 6828.78
Autokleider	$ 24303.05	$ 17044.67	$ 7258.38
Belgium Black Jeans	$ 44726.48	$ 0.00	$ 44726.48
Big Foot Shoes	$ 21255.19	$ 11606.34	$ 9648.85
Boleros	$ 214752.71	$ 191627.32	$ 23125.39
Bond Ltd	$ 95338.61	$ 51400.51	$ 43938.10
Boombastic	$ 119177.71	$ 58992.75	$ 60184.97
Casual Clothing	$ 45651.67	$ 17723.93	$ 27927.74
Champes	$ 120196.44	$ 114127.58	$ 6068.87